

Software

Engineering I

LECTURE 1: Software Engineering

Topics - Recap

- Introduction to Software Engineering
 - What is Software?
 - What is Software Engineering?
 - Why Software Engineering?
 - Why Software Engineering is Difficult?
 - Software Attributes.
 - What affects software engineering?
 - What are Software Fundamental Activities?
 - Software engineering ethics

What is software?

- Computer programs and associated documentation.
- Software products may be developed for a particular customer or may be developed for a general market.
- Software products may be
 - Generic - developed to be sold to a range of different customers e.g. PC software such as Excel or Word.
 - Custom - developed for a single customer according to their specification.
- New software can be created by developing new programs, configuring generic software systems or reusing existing software.

What is software?

- **Generic products** These are stand-alone systems that are produced by a development organization and sold on the open market to any customer who is able to buy them.
- Examples of this type of product include software for PCs such as databases, word processors, drawing packages, and project-management tools.

What is software?

- **Customized (or bespoke) products** - These are systems that are commissioned by a particular customer.
- A software contractor develops the software especially for that customer.
- Examples of this type of software include control systems for electronic devices, systems written to support a particular business process, and air traffic control systems.

What is Software Engineering?



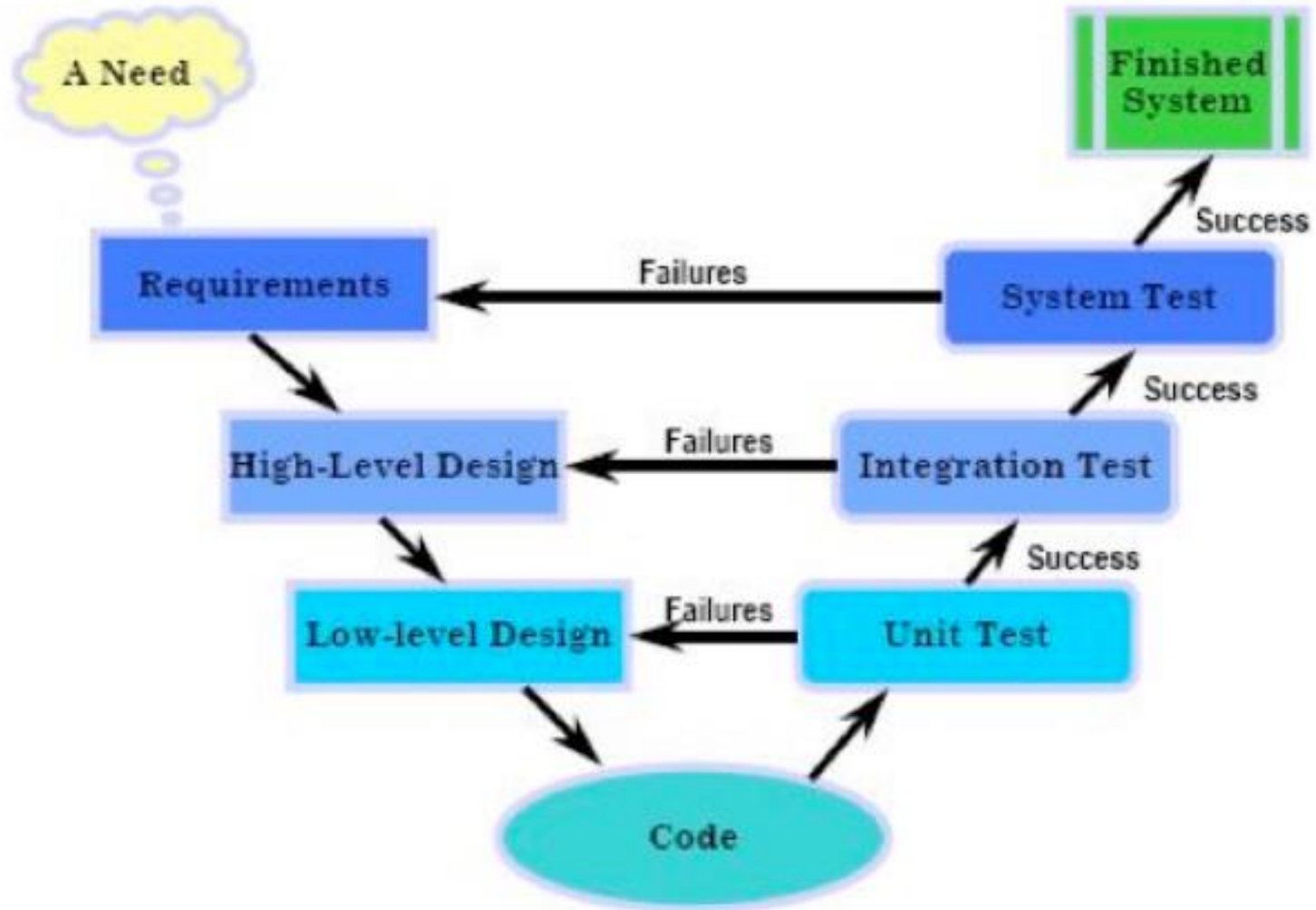
What is Software Engineering?

- Software engineering is an engineering discipline that is concerned with all aspects of software production from the early stages of system specification through to maintaining the system after it has gone into use.
- **Engineering discipline**, Engineers make things work.
- **All aspects of software production**, Software engineering is not just concerned with the technical processes of software development.
 - It also includes activities such as software project management and the development of tools, methods, and theories to support software production.

What is Software Engineering?

- Practical application of computer science, management techniques, and other skills to: design, construct, and maintain software and its documentation
- Systematic application of methods, tools, and techniques to achieve a stated requirement or objective for software system
- Application of systems engineering to software development
- Uses engineering discipline to reduce problems of late delivery, cost overruns, and failure to meet requirements
- A means for communicating amongst stakeholders

What is Software Engineering?



What is the difference between software engineering and computer science?

- Computer science is concerned with theory and fundamentals; software engineering is concerned with the practicalities of developing and delivering useful software.
- Software engineering is closer to good business practices than science
- Computer science theories are still insufficient to act as a complete underpinning for software engineering (unlike e.g. physics and electrical engineering).

What is the difference between software engineering and system engineering?

- System engineering is concerned with all aspects of systems development including hardware, software and process engineering.
- Software engineering is part of this process concerned with developing the software infrastructure, control, applications and databases in the system.
- System engineers are involved in system specification, architectural design, integration and deployment.

Why Software Engineering?



Why Software Engineering?

- Software affects nearly every aspect of our everyday lives and has become pervasive (universal) in our commerce, our culture, and our everyday activities.
- Software can have huge impact in any aspect of our society.

Why Software Engineering?

- The economies of ALL developed nations are dependent on software.
- More and more systems are software controlled
- Software engineering is concerned with theories, methods and tools for professional software development.
- Expenditure on software represents a significant fraction of GNP in all developed countries.

Why Software Engineering?

- Where can we find software?



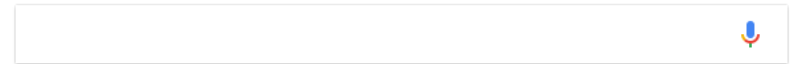
Why Software Engineering?

- Where can we find software?



Why Software Engineering?

- Where can we find software?
- Some popular ones...



Google Search

I'm Feeling Lucky



Software Costs

- Software costs often dominate computer system costs.
- The costs of software on a PC are often greater than the hardware cost.
- Software costs more to maintain than it does to develop. For systems with a long life,
- maintenance costs may be several times development costs.
- Software engineering is concerned with cost effective software development.

What are the costs of software engineering?

- Roughly 60% of costs are development costs, 40% are testing costs. For custom software, evolution costs often exceed development costs.
- Costs vary depending on the type of system being developed and the requirements of system attributes such as performance and system reliability.
- Distribution of costs depends on the development model that is used.

Why Software Engineering is Difficult?

- If you are developing software you need to know about **software domain** (because that is what you are building) and you need to know about the **problem domain** (because that is what you are building a solution for).
- A further problem is that software is a **formal domain**, where the inputs and goal states are well defined. BUT, the real world is **informal** with ill-defined inputs and goal states.
- Software engineering is mainly about modeling the physical world and finding good abstractions.

Why Software Engineering is Difficult?

- Our abstractions are unavoidably just approximations.
- We cannot describe the problem domain in perfect detail.
- We live in a **changing world**: things wear out and break, organizations go bankrupt or get acquired or restructured, business practices change, government regulations change, fads and fashions change, and so on.

Why Software Engineering is Difficult?

- Software development still largely depends on heroic effort of select few developers. Product line and development standardization are still largely missing, but there are efforts in this direction.

“Software is like entropy. It is difficult to grasp, weighs nothing, and obeys the second law of thermodynamics; i.e., it always increases.”

- Norman R. Augustine

Software Attributes

- Good software should strive to meet these requirements. It is not possible to meet all requirements to their fullest (e.g: higher security might mean less efficiency).
 - **Maintainability**
 - **Dependability and security**
 - **Efficiency**
 - **Acceptability**

Software Attributes

- Maintainability

Software should be written in such a way so that it can evolve to meet the changing needs of customers. This is a critical attribute because software change is an inevitable requirement of a changing business environment

- Dependability and security

Software dependability includes a range of characteristics including reliability, security, and safety. Dependable software should not cause physical or economic damage in the event of system failure. Malicious users should not be able to access or damage the system.

Software Attributes

- Efficiency

Software should not make wasteful use of system resources such as memory and processor cycles. Efficiency therefore includes responsiveness, processing time, memory utilization, etc.

- Acceptability

Software must be acceptable to the type of users for which it is designed. This means that it must be understandable, usable, and compatible with other systems that they use.

What affects software engineering?

- A lot of software is bad, this is due to a lot of factors, such as:
- **Increasing demand**: The world wants software, but sometimes faster than what anyone could possibly deliver.
- **Low expectations**: Developers aren't used to using good methods and develop poor software. Users are used to mediocre results.
- **Bad development methods**: Factors such as customer contracts (or simply incompetence or fear of novelty) prevents teams from choosing the appropriate development methods. Certain development methods, such as the waterfall method, is old; it has a lot of important uses, but agile methods might in many cases be better.

What are Software Fundamental Activities?

- A software process is a sequence of activities that leads to the production of a software product.
- There are four fundamental activities that are common to all software processes
 - **Software specification**
 - **Software development**
 - **Software validation**
 - **Software evolution**

What are Software Fundamental Activities?

- Software specification (What will it do?)

Customers and engineers define the software that is to be produced and the constraints on its operation.

- Software development (Making it do it.)

The software is designed and programmed.

What are Software Fundamental Activities?

- Software validation (Does it do it?)

The software is checked to ensure that it is what the customer requires

- Software evolution

The software is modified to reflect changing customer and market requirements.

Modifying the software after release to reflect new requirements.

Professional and ethical responsibility

- Software engineering involves wider responsibilities than simply the application of technical skills.
- Software engineers must behave in an honest and ethically responsible way if they are to be respected as professionals.
- Ethical behaviour is more than simply upholding the law.

Software engineering ethics

- Like other engineering disciplines, software engineering is carried out within a social and legal framework that limits the freedom of people working in that area.
- A Software Engineer must also behave in an ethical and morally responsible way if you are to be respected as a professional engineer.
- A Software Engineer should not use your skills and abilities to behave in a dishonest way or in a way that will bring disrepute to the software engineering profession.

Software engineering ethics

- A Software Engineer should always follow these ethical guidelines.

- 1. Confidentiality**
- 2. Competence**
- 3. Intellectual property rights**
- 4. Computer misuse**

Software engineering ethics

- Confidentiality

A Software Engineer should normally respect the confidentiality of your employers or clients irrespective of whether or not a formal confidentiality agreement has been signed.

Often you might sign a confidentiality agreement, but not necessarily.

Treat your customer's information as **secret**.

Software engineering ethics

- Competence

A Software Engineer should not misrepresent your level of competence. You should not knowingly accept work that is outside your competence.

Software engineering ethics

- Intellectual property rights

A Software Engineer should be aware of local laws governing the use of intellectual property such as **patents** and **copyright**.

A Software Engineer should be careful to ensure that the intellectual property of employers and clients is protected.

Software engineering ethics

- Computer misuse

A Software Engineer should not use your technical skills to misuse other people's computers.

Computer misuse ranges from relatively trivial (game playing on an employer's machine, say) to extremely serious (dissemination of viruses or other malware).

IEEE Computer Society Code of Ethics

- The professional societies in the US have cooperated to produce a code of ethical practice.
- Members of these organisations sign up to the code of practice when they join.
- The Code contains eight Principles related to the behaviour of and decisions made by professional software engineers, including practitioners, educators, managers, supervisors and policy makers, as well as trainees and students of the profession.

Code of ethics - preamble

- Software engineers shall commit themselves to making the analysis, specification, design, development, testing and maintenance of software a beneficial and respected profession. In accordance with their commitment to the health, safety and welfare of the public, software engineers shall adhere to the following Eight Principles:

Code of ethics - preamble

- Software engineers shall commit themselves to making the analysis, specification, design, development, testing and maintenance of software a beneficial and respected profession. In accordance with their commitment to the health, safety and welfare of the public, software engineers shall adhere to the following Eight Principles:

Code of ethics - principles

1. PUBLIC

- Software engineers shall act consistently with the public interest.

2. CLIENT AND EMPLOYER

- Software engineers shall act in a manner that is in the best interests of their client and employer consistent with the public interest.

3. PRODUCT

- Software engineers shall ensure that their products and related modifications meet the highest professional standards possible

Code of ethics - principles

4. JUDGMENT

- Software engineers shall maintain integrity and independence in their professional judgment.

5. MANAGEMENT

- Software engineering managers and leaders shall subscribe to and promote an ethical approach to the management of software development and maintenance.

6. PROFESSION

- Software engineers shall advance the integrity and reputation of the profession consistent with the public interest.

Code of ethics - principles

7. COLLEAGUES

- Software engineers shall be fair to and supportive of their colleagues.

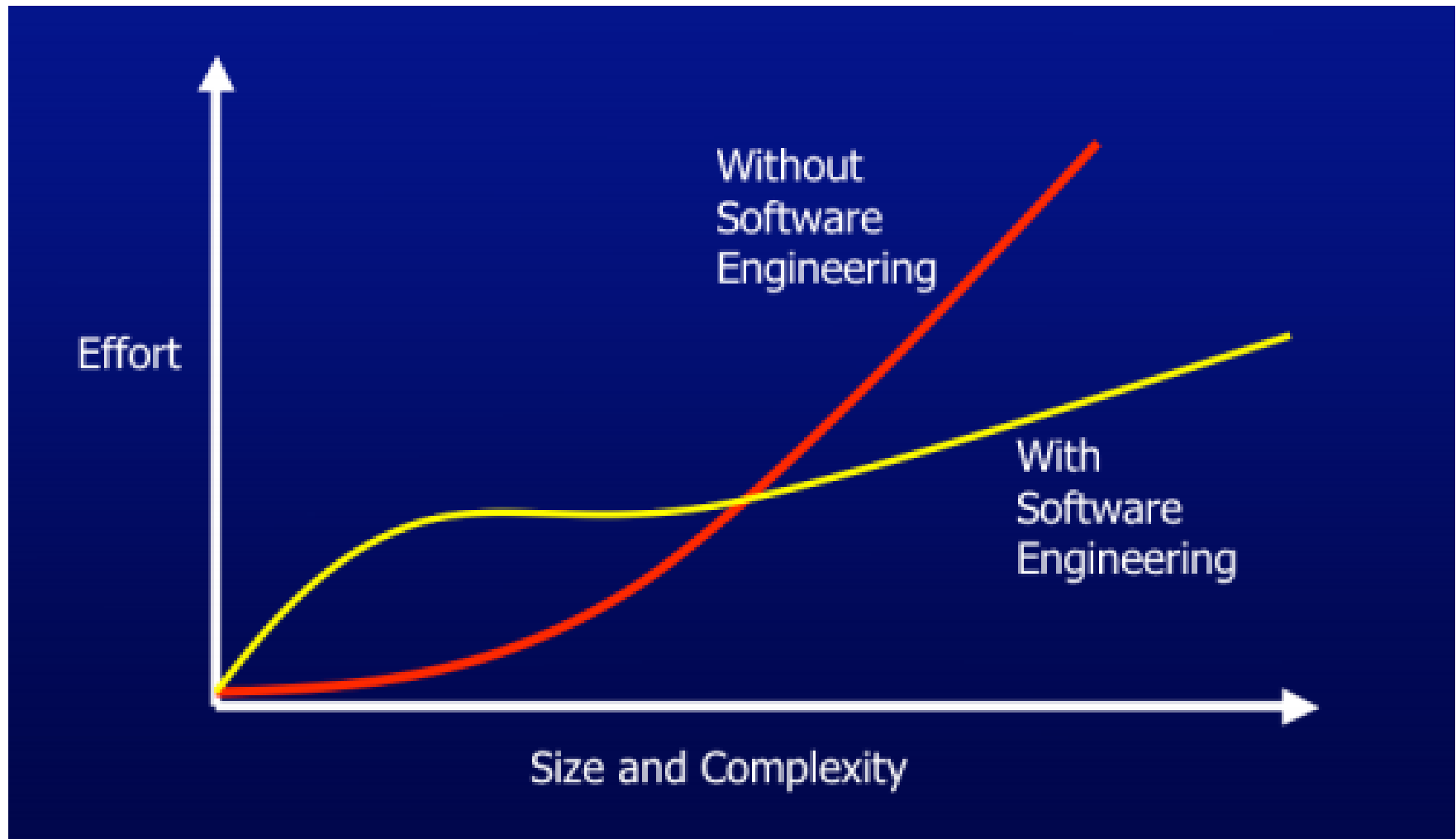
8. SELF

- Software engineers shall participate in lifelong learning regarding the practice of their profession and shall promote an ethical approach to the practice of the profession.

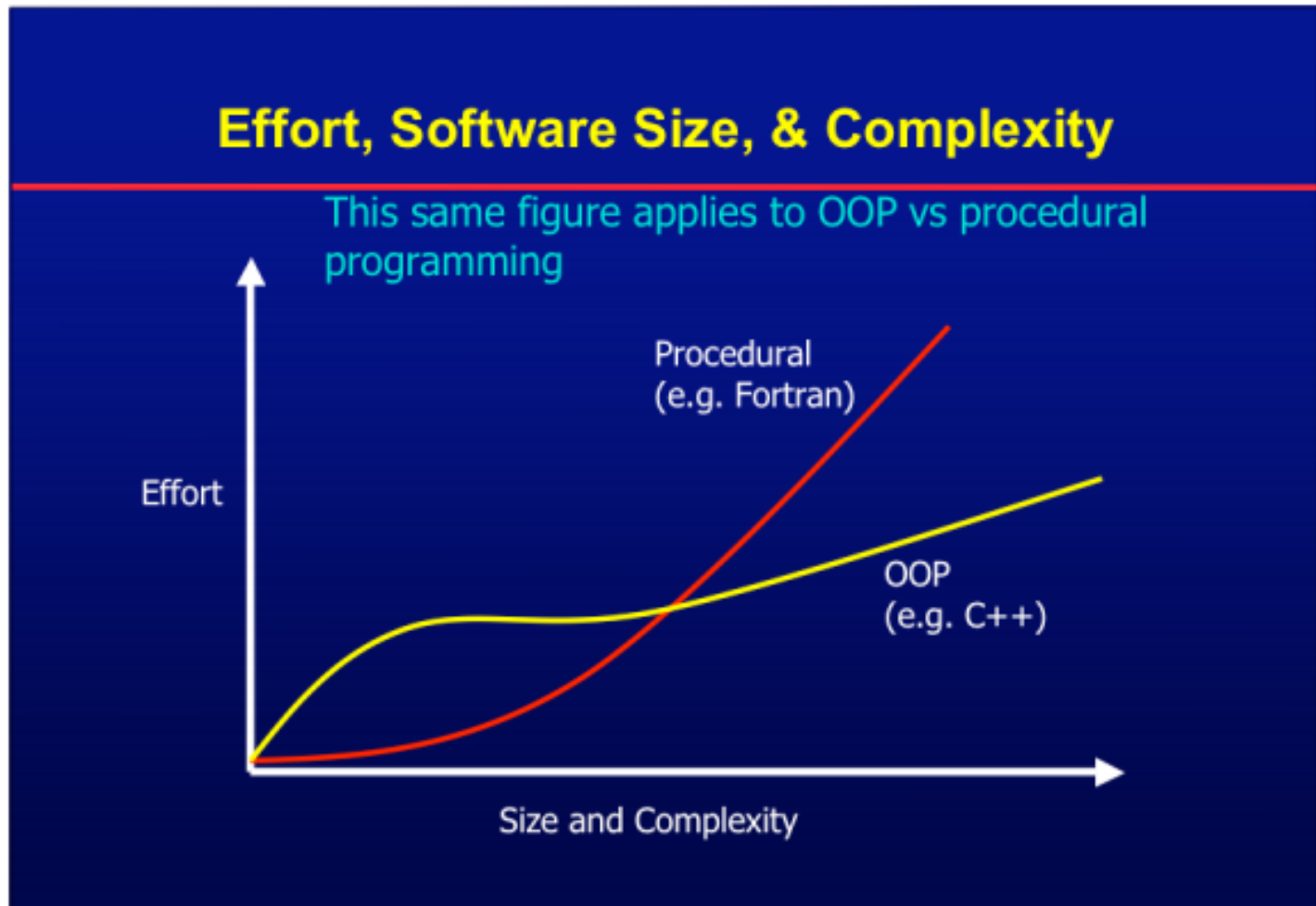
Ethical dilemmas

- Disagreement in principle with the policies of senior management.
- Your employer acts in an unethical way and releases a safety-critical system without finishing the testing of the system.
- Participation in the development of military weapons systems or nuclear systems

Effort, Software Size, & Complexity



Effort, Software Size, & Complexity



Key points

- Software engineering is an engineering discipline that is concerned with all aspects of software production.
- Software products consist of developed programs and associated documentation. Essential product attributes are maintainability, dependability, efficiency and usability.
- The software process consists of activities that are involved in developing software products. Basic activities are software specification, development, validation and evolution.
- Methods are organised ways of producing software. They include suggestions for the process to be followed, the notations to be used, rules governing the system descriptions which are produced and design guidelines.

Key points

- Software engineers have responsibilities to the engineering profession and society. They should not simply be concerned with technical issues.
- Professional societies publish codes of conduct which set out the standards of behavior expected of their members.

Key points

- Software engineering developed out of necessity to handle large software projects that cannot be handled by few individuals using ad hoc methods
- Goal is to develop software systems that:
 - Satisfy technical requirements, user needs, and acquirer expectations
 - Are developed on time and on budget
 - Are easy to modify and maintain
 - Are developed with pride and personal satisfaction
 - Fulfills all ethical and legal considerations
- Good project management and software engineering processes are required

The End